EE 250L: Distributed Systems for the Internet of Things

Fall 2026 | Lab 6

Interfacing Sensors with the ESP32-S3

	Tuesday Lab	Thursday Lab
Assigned Date	10/13/26	10/15/26
Submission Due Date	10/19/26 (3 PM)	10/21/26 (3 PM)
Interview + Demo Due	10/20/26 (Before Lab)	10/22/26 (Before Lab)

Lab slides: https://claude.ai/artifact/RK2BubmhTeesUK6PVNkCqm

Lab outcome:

1. Build five sensor/actuator circuits on a breadboard and wire them to an ESP32-S3-DevKitC-1: a light sensor, a temperature sensor, a vibration switch, a push button and an LED.
2. Read the two analog sensors with the ESP32-S3's on-chip ADC and the two switches as digital inputs, and drive the LED, writing the firmware yourself in ESP-IDF.
3. Verify every branch works by running a test routine and reading the serial monitor.

Teams: This lab is done in **teams of two (mandatory)**. You build and test the circuit together, but **each student submits individually**: your own zip with your own lab report, listing your name and your teammate's name.

Files to submit: `sensor_board.c` (your firmware), a labeled breadboard photo, `serial_output.txt` (the monitor output of your test routine), and your lab report `lab6_report.pdf` (screenshots of your test routine running, answers to the questions in Section 5 and acknowledgment of assistance). The lab report **must be a PDF**. See Section 7.

1. Introduction

In this lab you will interface a set of real sensors to a microcontroller and prove, in firmware, that the hardware works. You will build five small circuits on a breadboard, wire them to an ESP32-S3-DevKitC-1, and read light, temperature, vibration and a push button while driving an LED — with no shield and no external ADC.

Along the way you will practice the core skills of embedded hardware bring-up: turning a sensor into a voltage the ADC can read, giving a digital input a defined logic level, limiting current into an output, and using a multimeter and the serial monitor to find wiring faults. The circuit you validate here is the one you lay out as a PCB later in the course, so a clean, tested prototype now saves rework.

By the end of this lab you can: read an analog sensor with an on-chip ADC by building a voltage divider and converting raw counts to a physical quantity; read a digital input with a pull-up resistor and handle a bouncing contact by counting edges; drive an LED with the right series resistor and orientation; and write ESP-IDF firmware that samples sensors on a fixed cadence and acts on the readings.

2. The sensor board

The board is five independent branches between 3V3 and GND, each with one node wired to a GPIO through header J1. Figure 1 is the schematic.

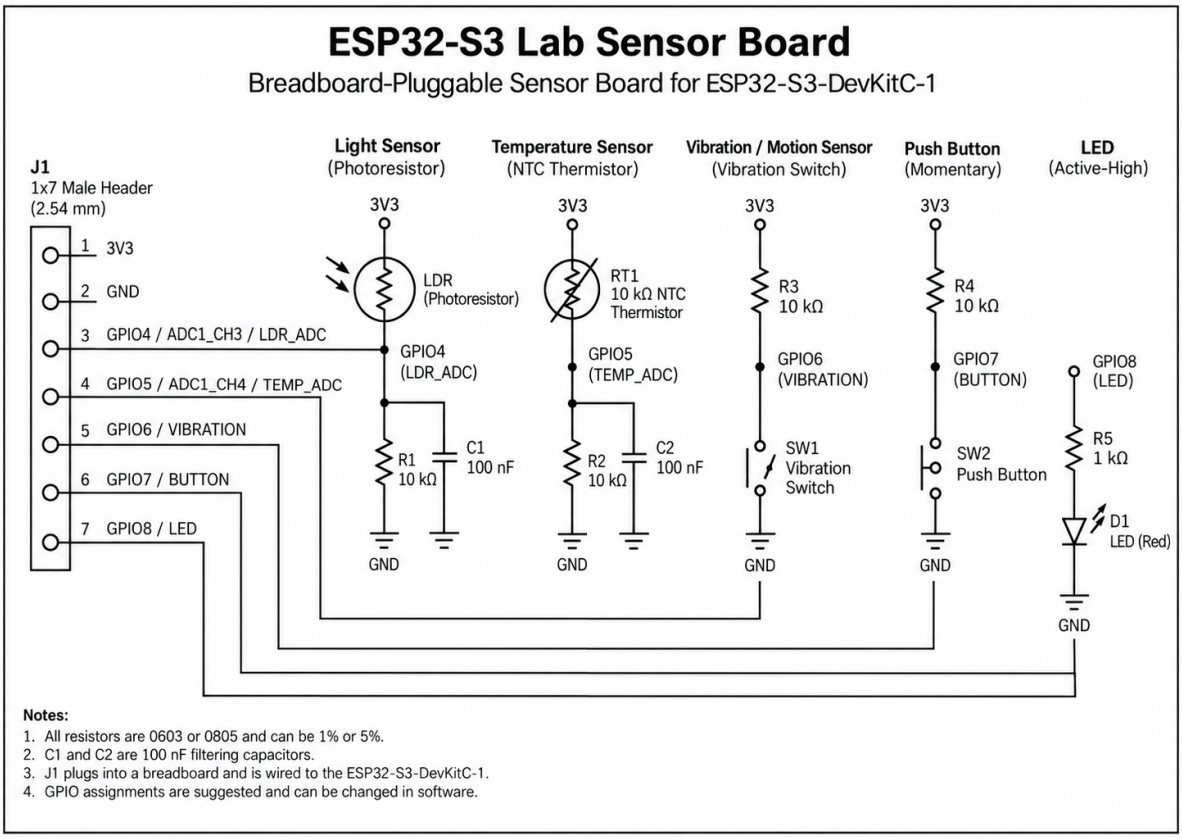


Figure 1: The sensor board schematic. Five branches between 3V3 and GND; each node connects to one GPIO through header J1. The part values in Table 2 are the ones to use: RT1 is a 100 kΩ thermistor and R2 is 100 kΩ.

2.1 Analog vs digital, and why no external ADC

Many sensors produce analog signals that must be converted to digital before a processor can use them. Some platforms have no ADC (the Raspberry Pi needs an external MCP3008 over SPI); the ESP32-S3 has two 12-bit ADCs on-chip, so analog sensors connect straight to its GPIO pins. Other sensors are already digital — a switch is open or closed, read as 1 or 0. This board mixes both. Table 1 lists the header connections, and Figure 2 shows where those pins are on the DevKitC-1.

Table 1: Pin connection table (header J1 to the DevKitC-1). Every sensor runs at 3.3 V; there is no 5 V line.

J1 pin	Signal	ESP32-S3 pin	Role
1	3V3	3V3	Power to all branches
2	GND	GND	Common ground
3	LDR_ADC	GPIO4 / ADC1_CH3	Light-sensor divider node
4	TEMP_ADC	GPIO5 / ADC1_CH4	Thermistor divider node
5	VIBRATION	GPIO6	Vibration switch, active-low
6	BUTTON	GPIO7	Push button, active-low
7	LED	GPIO8	LED drive, active-high

Facts you need for the design. The ADC returns 12-bit values (0–4095) over roughly 0–3.1 V, so each analog branch is a voltage divider from 3V3. Digital inputs need a defined level when the switch is open, so each switch has a 10 kΩ pull-up and reads 0 when closed.

2.2 The ESP32-S3-DevKitC-1 pinout

Figure 2 shows the two 22-pin headers of the DevKitC-1 and what each pin can do. Hold the board with the antenna away from you and the USB connectors toward you: the left header then starts with 3V3 and the right header with G. The seven pins this lab uses (Table 1) are outlined in gold. They are all on the left header, so only that header has to be plugged into the breadboard (Figure 9). The pin names are printed on the board beside each header — wire by the printed name, not by counting.

How to read the figure:

- **Power (3V3, 5V, G).** 3V3 powers every branch in this lab and all G pins are the same ground. The 5V pin carries USB voltage; the GPIOs are 3.3 V only, so never connect 5V to one.
- **Analog-capable pins.** GPIO1–GPIO10 are channels of ADC1 and are the right choice for analog sensors. GPIO11–GPIO20 are channels of ADC2, which is shared with the Wi-Fi driver, so its readings can fail while Wi-Fi is on.
- **Digital pins.** Every GPIO can be a digital input or output; the ones marked “digital only” have no ADC channel.
- **Reserved pins.** The strapping pins (GPIO0, 3, 45, 46) are sampled at reset to configure how the chip boots; TX and RX carry the serial monitor; GPIO19 and GPIO20 are the native USB data lines; GPIO35–GPIO37 are wired to the module’s memory on some board variants. Leave these unconnected unless you know you need them.

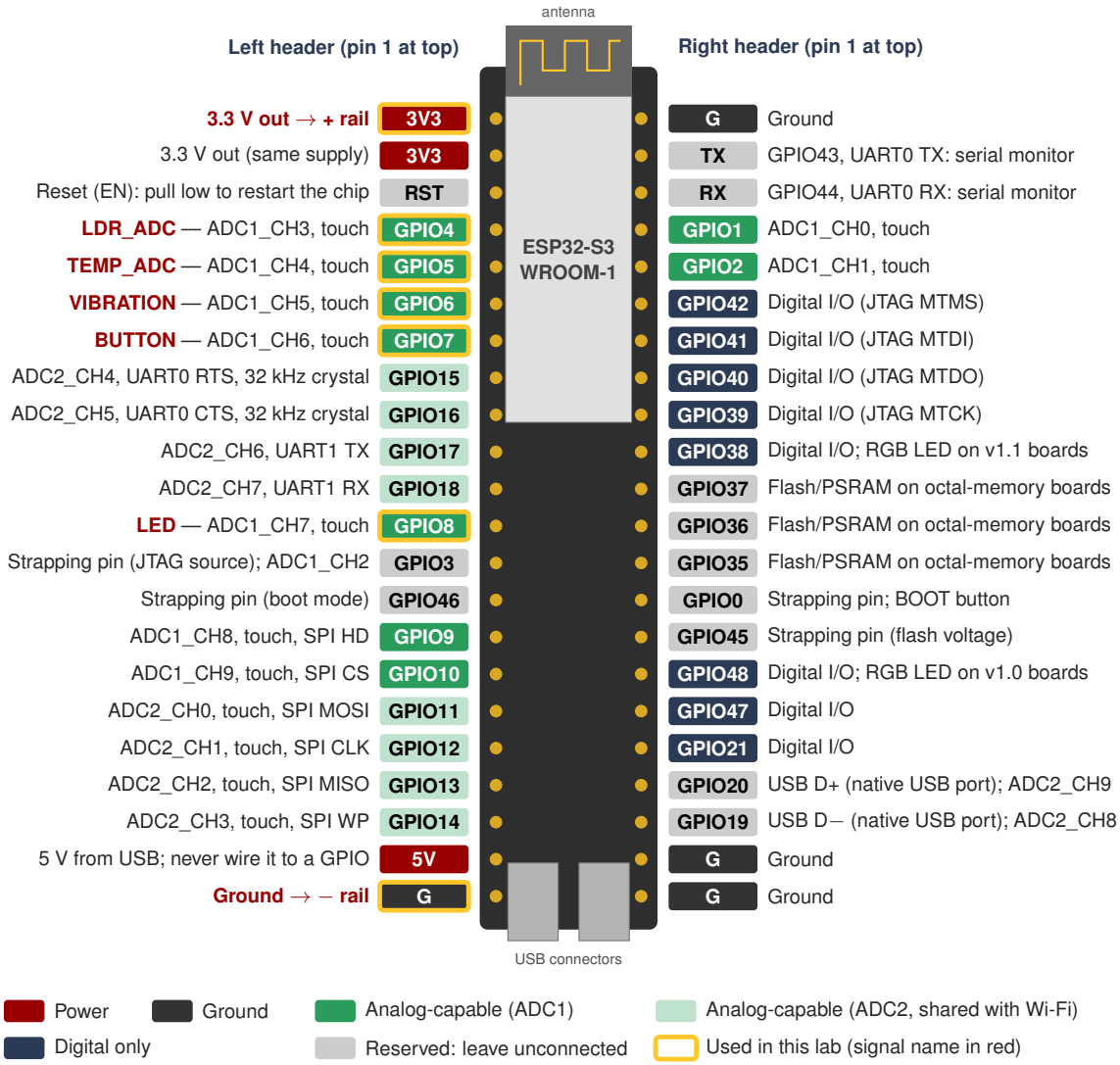


Figure 2: ESP32-S3-DevKitC-1 pinout, seen from above with the antenna at the top. Every pin is 3.3 V logic. The drawing is schematic: it shows pin order and function, not the exact board artwork.

2.3 The five branches

Table 2: Bill of materials and the circuit of each branch.

Branch	Parts	Circuit	What the GPIO sees
Light sensor	LDR (GL55), R1 10 kΩ, C1 100 nF	3V3 → LDR → node → R1 → GND	Voltage rises with light.
Temperature	RT1 NTC 100 kΩ@25 °C, R2, C2 100 nF	3V3 → RT1 → node → R2 → GND	Voltage rises as it warms; convert with the β equation.
Vibration	R3 10 kΩ, SW1 SW-520D ball switch	3V3 → R3 → node → SW1 → GND	Rests at 1; the contact chatters when tapped.
Push button	R4 10 kΩ, SW2 momentary	3V3 → R4 → node → SW2 → GND	1 released, 0 pressed.
LED	R5 1 kΩ, D1 red LED	GPIO8 → R5 → D1 → GND	Driving the pin high lights the LED.

Figure 3 shows what each part looks like, so you can pick the right one out of your kit.



Figure 3: The parts used in this lab

Analog branches: voltage dividers. The ADC measures voltage, not resistance, so each analog sensor forms a divider with a fixed resistor and the ADC reads the node between them (Figure 4).

Pick the fixed resistor so the node sits near mid-scale at the operating point (Figure 5): the thermistor measures about 113 kΩ at room temperature, so it is a 100 kΩ part and R2 should be 100 kΩ.

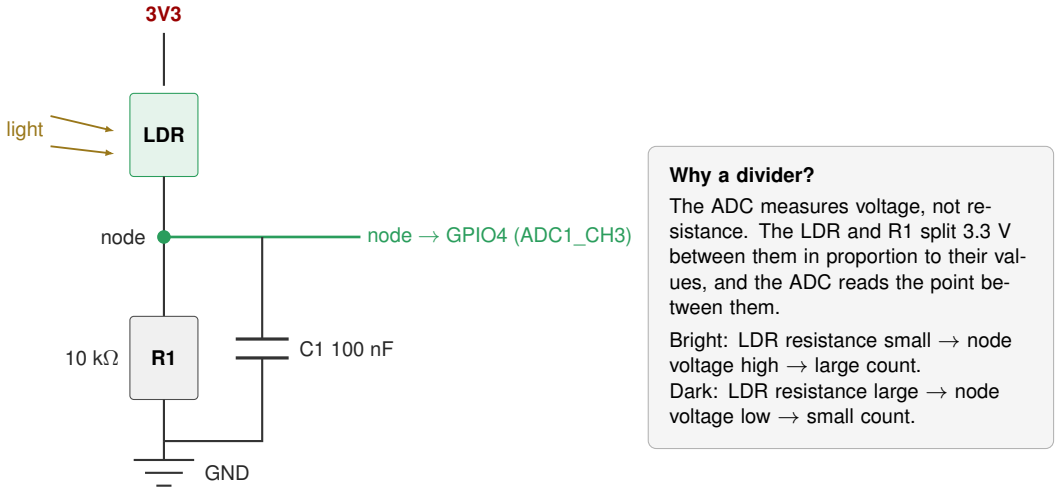


Figure 4: The light-sensor branch. The ADC reads the voltage at the node between the LDR and R1.

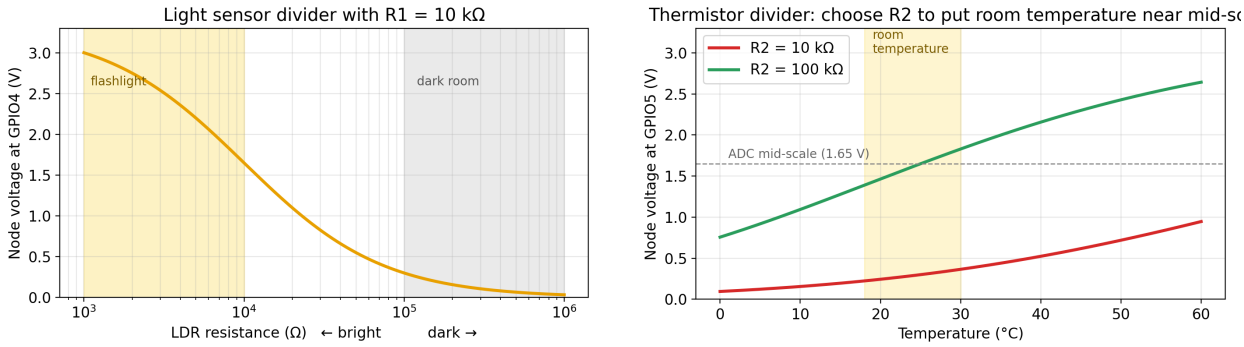


Figure 5: Left: LDR divider node voltage over its resistance range. Right: thermistor node voltage vs temperature with R2 = 100 kΩ; room temperature sits near mid-scale, where the ADC resolves best.

Digital branches: pull-ups and active-low inputs. R3 and R4 hold their nodes at 3.3 V while the switch is open and limit the current to 0.33 mA when it closes (Figure 6). The input reads 1 at rest and 0 when the switch closes — active-low.

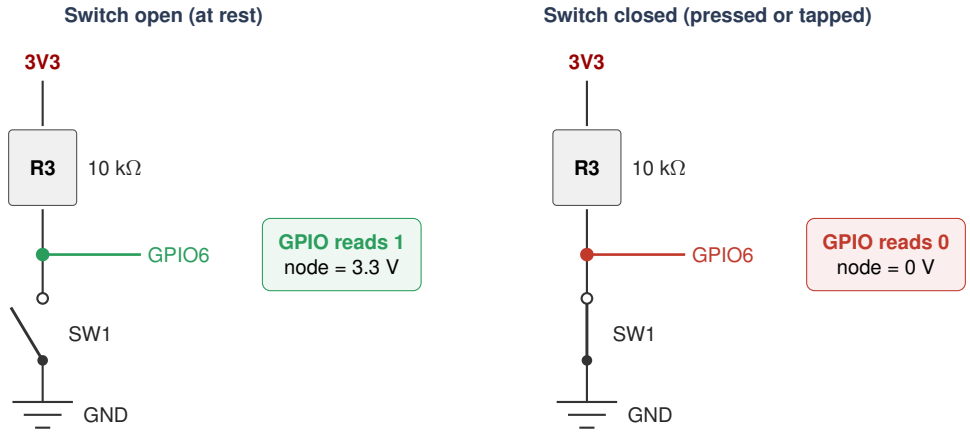


Figure 6: Pull-up on the vibration branch. The button branch (R4, SW2) is identical.

Capacitors go on the analog branches only. C1 and C2 filter noise on the two ADC nodes. A capacitor on the vibration branch would smooth out the contact bounce the firmware relies on to detect a tap — do not add one there.

The SW-520D is a metal ball in a can; a tap makes its contact chatter, so vibration is detected by counting edges rather than reading a level.

Output branch: the LED. GPIO8 drives the LED through R5 (Figure 7). The longer leg is the anode and faces the GPIO; backwards, the LED blocks current and stays dark.

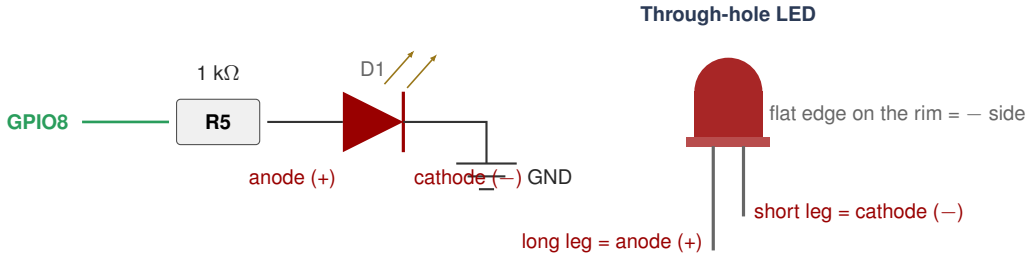


Figure 7: The LED branch. R5 limits the current through D1. The longer leg is the anode and faces the GPIO.

3. Part 1: Prototype the circuit on a breadboard

Before the PCB is manufactured, prototype the circuit on a breadboard. Figure 8 shows how a breadboard is wired inside; Figure 10 is the layout to build.

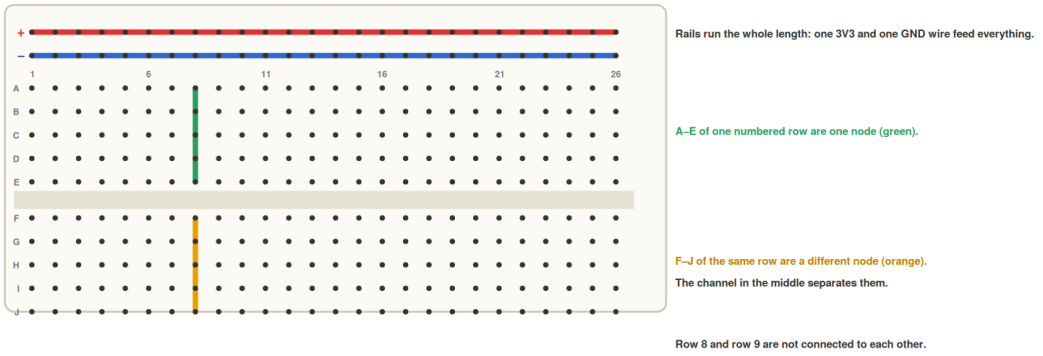


Figure 8: Breadboard connectivity. Two legs connect only when they share a numbered row on the same side of the center channel; the + and − rails run the whole length.

Placing the ESP32-S3. Figure 10 shows the wiring with the screw-terminal adapter for the DevKitC-1. If you have the adapter, wire it as shown there. If you do not, plug the board itself into the breadboard: the DevKitC-1 does not fit across the breadboard, and it does not need to, because every pin this lab uses (3V3, GPIO4–GPIO8 and G) is on the same header, the one that starts with 3V3. Plug **only that header** into the breadboard and let the other header hang off the edge, in the air, as in Figure 9. Take ground from the G pin on that same header (next to 5V, at the USB end), and keep the pins that are in the air away from anything metal.

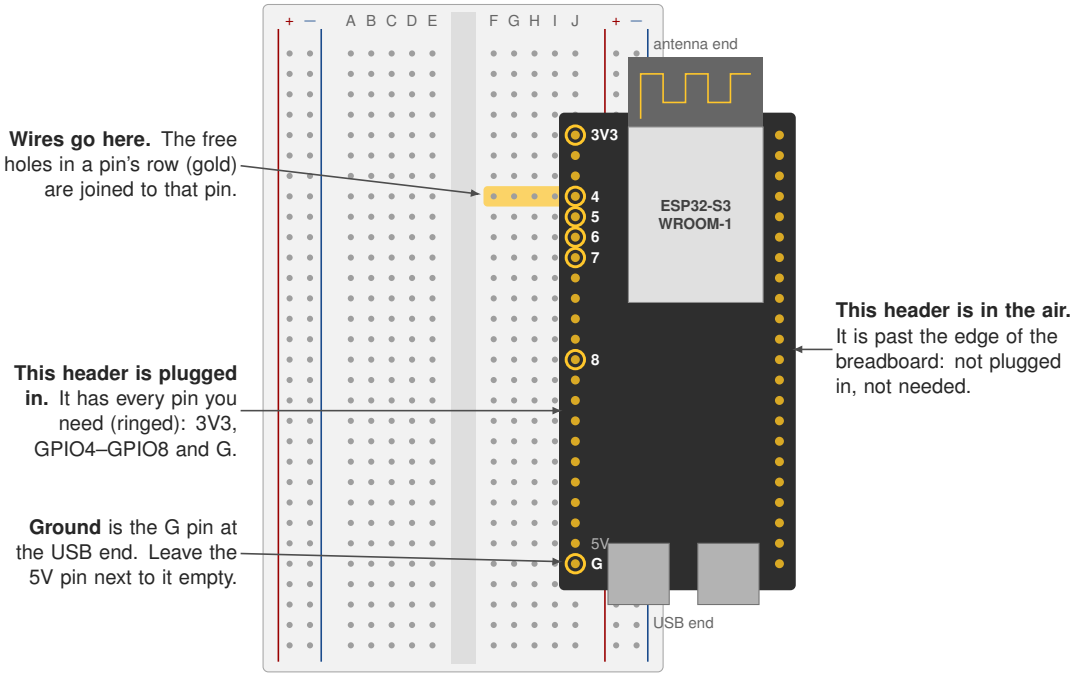


Figure 9: Where the ESP32-S3 goes (schematic drawing, seen from above). Only the header with 3V3, GPIO4–GPIO8 and G is plugged into the breadboard; the other header hangs off the edge.

Build order.

1. Wire 3V3 → + rail and GND → − rail. Check with a multimeter that each rail is continuous end to end (some full-size boards split in the middle).

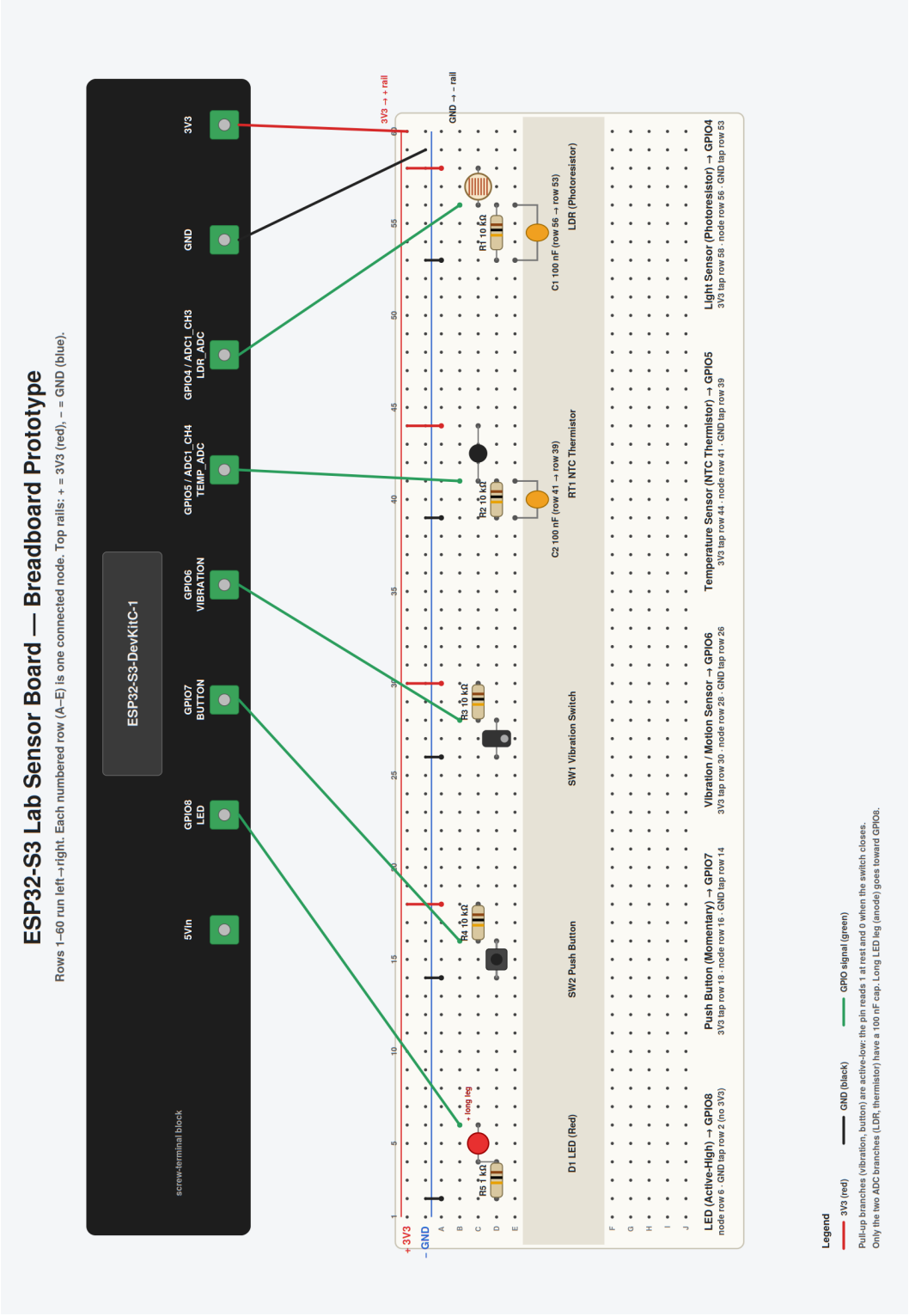


Figure 10: Breadboard layout for the sensor board, shown with the screw-terminal adapter for the DevKitC-1. Each branch uses a 3V3 tap row, a node row (to the GPIO) and a GND tap row; row numbers are printed under each branch. Turn the page to read it.

2. For each branch, run a short red jumper from + into its 3V3 tap row and a short black jumper from − into its GND tap row.
3. Place the parts between the rows as in Figure 10. Keep each branch on the A–E side.
4. Run one wire from each node row to its GPIO terminal (GPIO4–GPIO8); Figure 2 shows where each one is on the DevKitC-1.

Before powering up. Keep wires short and use a color code (red 3V3, black GND, green signals). Check that every leg sits in the row it belongs to, that the LED faces the right way, and with the multimeter that 3V3 is not shorted to GND.

4. Part 2: Testing the hardware with software

Once the PCB comes back from the manufacturer you will need to test it, so write firmware now that reads every sensor and drives the LED. Use ESP-IDF in C, with the environment from Lab 3. In every new terminal load it, then create and build the project:

```
. ~/esp/esp-idf/export.sh          # load the ESP-IDF environment (as in Lab 3)
cd ~/esp
idf.py create-project sensor_board
cd sensor_board
idf.py set-target esp32s3
# write your code in main/sensor_board.c, then:
idf.py build
idf.py -p <PORT> flash monitor     # Ctrl+] closes the monitor
```

Replace `<PORT>` with your board's serial port: `ls /dev/cu.*` (macOS, e.g. `/dev/cu.usbmodem1101`), `ls /dev/ttyUSB* /dev/ttyACM*` (Linux), or Device Manager → Ports (Windows). Use the port labeled UART. If flashing says the port does not exist, re-check the name after re-plugging; if it says no serial data received, hold BOOT, tap RESET, release BOOT and retry.

You write the firmware yourself. The ESP-IDF ADC oneshot and GPIO references have everything you need: the oneshot ADC driver for the two analog branches (ADC_UNIT_1, 12 dB attenuation, 12-bit width), the GPIO driver for the switches and the LED, a GPIO interrupt on any edge to count vibration bounces, and `vTaskDelayUntil` for a fixed sampling period. Give each sensor its own read function so you can reuse it later.

4.1 Test routine

When the board starts, **blink the LED 5 times** with 500 ms on/off intervals. Then run an infinite loop that, every 100 ms, reads **all four sensors** and prints them on **one line**, so you can watch every branch at the same time:

```
light=<raw> (bright|dark)  temp_raw=<raw> (<T> C)  button=PRESSED|released  vibration=<n>
```

Each line contains:

1. **Light:** the raw ADC value with the text "bright" or "dark". Find the threshold yourself by experiment (cover the LDR, then shine a phone flashlight on it).
2. **Temperature:** the raw ADC value and the temperature in °C. Pinch the thermistor; the temperature should rise a few degrees.
3. **Button:** "PRESSED" or "released".
4. **Vibration:** the number of edges counted since the previous line.

After the startup blinks, the LED **turns on while the button is pressed**. It also turns on whenever the vibration edge count on a line exceeds a threshold you choose. Keep the 100 ms cadence fixed.

For the thermistor, convert the reading with the divider and the β equation (RT1 from 3V3 to the node, R2 from the node to GND):

$$R_{\mathrm{NTC}} = R_2\left(\frac{4095}{\mathrm{raw}} - 1\right), \qquad \frac{1}{T} = \frac{1}{298.15} + \frac{1}{\beta}\ln\frac{R_{\mathrm{NTC}}}{R_{25}},$$

with $R_{25} = 100\ \mathrm{k\Omega}$, $\beta = 3950$ and T in kelvin.

Take screenshots as you test. Screenshot the serial monitor while the test routine runs; the screenshots go in your lab report (Section 5).

5. Lab Report

Each student submits their own lab report. It contains, in this order:

1. Your name and your teammate's name.
2. **Screenshots of your test routine** running in the serial monitor (Section 4.1).
3. Your answers to the questions below, numbered to match.
4. Your acknowledgment of assistance (AI tools, people, websites).

Submit the lab report as a PDF named `lab6_report.pdf`, not as a `README.md` or any other file type.

5.1 Questions

1. Why do the switch branches need a pull-up resistor? What would the GPIO read without it?
2. The LDR branch puts the LDR on the 3V3 side and R1 on the GND side. What changes in the ADC reading if you swap them?
3. With R2 = 10 kΩ, roughly what voltage does the thermistor node sit at near room temperature, and why is that a poor operating point for a 12-bit ADC?
4. What is the purpose of C1 and C2, and why must there be no capacitor on the vibration branch?
5. The vibration switch reads 1 when lying flat in one orientation and 0 in another. Why does edge counting work regardless of orientation?
6. What is the maximum current through the LED, and how did you compute it? Assume a forward voltage of about 1.8 V for the red LED.
7. The two analog sensors are on GPIO4 and GPIO5. Using Figure 2, explain why they could not be moved to GPIO42 and GPIO41, and why ADC1 pins are preferred over ADC2 pins.

6. Demo and Code Grading Rubric

Points	Description
15 (Demo)	All five branches working live: the LED blinks 5 times at startup, then all sensors are shown on one line of serial output: light reads bright/dark, thermistor reads a sensible temperature that rises when pinched, vibration is detected on a tap and lights the LED, the button reads pressed/released and lights the LED.
10 (Demo)	Clean breadboard build: correct dividers and pull-ups, capacitors only on the analog branches, LED oriented correctly, no short from 3V3 to GND.
5 (Submission)	Complete firmware, labeled photo, serial output and a PDF lab report with test-routine screenshots and answers to all questions in Section 5, plus an explicit acknowledgment of all assistance (AI tools, people, websites).
70 (Interview)	Answer questions posed by the TA/CP about this lab to their satisfaction, showing you understood each part and did not just do it mechanically.
100	**Total**

7. Submission

Each student submits their own zip via Brightspace, containing:

1. `sensor_board.c` — your firmware from Section 4.
2. A labeled photo of your breadboard from Section 3.
3. `serial_output.txt` — the monitor output of your test routine (Section 4.1).
4. `lab6_report.pdf` — your own lab report as described in Section 5: your name and your teammate's name, screenshots of your test routine, your answers to the questions, and an acknowledgment of assistance.

The lab report must be a PDF.